For Loops

For Loops

- A for loop executes its statements a set number of times
  - a set number of iterations based on a numeric value
  - iterated through the object
    - lists
    - strings
    - dictionary
    - tuple
    - set (one way to iterate through)

Range

- we can create a range object and iterate through that
  - container object that contains integers of given specifications
  - range(start, stop, stride)
    - Look familiar?
    - works like slicing!
- This allows for the iteration a set number of times

For Each

- same general format of a for loop using range, but with a different object instead
  - string
  - list
  - tuple
  - dictionary
  - set

For Each

- iterates through the container or sequence type one element at a time
- does not use index, directly access the stored object

Range vs. For Each

- need numerical values of indexes
  - best for making modifications to container
- need element specifically
  - viewing or accessing, but not modifying

Nesting Loops

- loops can be placed in one another
- for every full iteration of internal loop, one iteration of external loop occurs
  - think of a clock

Break & Continue

- Break can be used to exit a loop immediately
- Continue can be used to jump to the next iteration immediately
  - sometimes used to skip certain values during iteration for one reason or another

Loop Else

- Loops may have an else statement
  - This occurs when a while loop or for loop completes normally (not using break)

Enumerate

- utilize index and value
- utilizes unpacking
  - extracting a container into variables